Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №2

Численные методы первого порядка для поиска безусловного экстремума
тема

Преподаватель _____ В.В. Тынченко
подпись, дата — инициалы, фамилия

Студент КИ24-17/1Б, 032428518 _____ Э. А. Алиев
номер группы, зачетной книжки — подпись, дата — инициалы, фамилия

Красноярск 2026

1. Постановка задачи

Требуется программно реализовать численные методы безусловной оптимизации варианта 1, проверить правильность работы алгоритмов на примерах из учебника А. В. Пантелеева и Т. А. Летовой и провести вычислительные эксперименты. Точность оценивается абсолютной погрешностью найденного минимального значения функции относительно реального минимума, который найден самостоятельно (аналитически) до проведения экспериментов. Скорость оценивается количеством вычислений целевой функции.

Для варианта 1 заданы методы (метод градиентного спуска с постоянным шагом, метод наискорейшего градиентного спуска) и функции двух переменных (рисунок 1 и 2 соответственно):

$$f(x) = (x_1 - 4)^2 + (x_2 - 1)^2 \to min$$

Рисунок 1 – Первая функция

$$f(x) = 2x_2^2 - 2x_2 + x_1x_2 + 4x_1^2 \to min$$

Рисунок 2 – Вторая функция

Для каждого исследуемого параметра необходимо построить графики зависимости погрешности и количества вычислений функции, дать краткий комментарий и представить итоговые результаты в сводной таблице, построенной программно.

2. Краткое описание методов

2.1. Метод градиентного спуска с постоянным шагом

На каждой итерации точка смещается против направления градиента функции на фиксированную длину шага alpha: $x_{k+1} = x_k - \text{alpha} \cdot \nabla f(x_k)$. Если alpha выбран слишком большим (превышает $2/\lambda_{max}$, где λ_{max} — наибольшее собственное число матрицы Гессе), метод расходится; если alpha слишком мал — сходимость становится очень медленной. Итерации останавливаются, когда норма градиента становится меньше заданной точности eps.

2.2. Метод наискорейшего градиентного спуска

Отличается от предыдущего метода тем, что на каждой итерации длина шага alpha_k не фиксирована, а выбирается так, чтобы минимизировать функцию вдоль направления антиградиента (одномерная задача линейного поиска). Поскольку обе заданные функции квадратичные, оптимальный шаг находится в явном виде: $\text{alpha}_k = (g^T g)/(g^T H g)$, где $g = \nabla f(x_k)$, H — матрица Гессе. Это избавляет метод от необходимости подбирать параметр alpha вручную, но для существенно «вытянутых» (плохо обусловленных) квадратичных функций траектория спуска приобретает характерный зигзагообразный вид и сходимость замедляется.

2.3. Определение точных минимумов функций

Обе функции варианта являются квадратичными, поэтому точный минимум находится аналитически из условия $\nabla f(x) = 0$.

Для $f_1(x_1, x_2) = (x_1 - 4)^2 + (x_2 - 1)^2$ градиент равен $(2(x_1 - 4), 2(x_2 - 1))$ и обращается в ноль при $x^* = (4, 1)$; матрица Гессе $H_1 = \text{diag}(2, 2)$ положительно определена, значит это точка глобального минимума, $f_1(x^*) = 0$.

Для $f_2(x_1, x_2) = 4x_1^2 + x_1x_2 + 2x_2^2 - 2x_2$ градиент равен $(8x_1 + x_2, x_1 + 4x_2 - 2)$. Решение линейной системы $8x_1 + x_2 = 0$, $x_1 + 4x_2 - 2 = 0$ даёт $x^* = (-2/31, 16/31)$

≈ (−0,064516, 0,516129), $f_2(x^*) \approx -0{,}516129$. Матрица Гессе $H_2 = [[8, 1], [1, 4]]$ имеет собственные числа $\lambda_1 \approx 3{,}764$ и $\lambda_2 \approx 8{,}236$, оба положительны, значит H_2 положительно определена и найденная точка — глобальный минимум. Из $\lambda_{max} \approx 8{,}236$ следует, что метод с постоянным шагом сходится для f_2 только при $alpha < 2/\lambda_{max} \approx 0{,}2428$ — это учтено при выборе диапазона варьирования шага в экспериментах. В отличие от f_1, функция f_2 содержит линейное слагаемое $(-2x_2)$, поэтому её минимум не совпадает с началом координат..

3. Текст программы с комментариями

Программа написана на Python. Для таблиц используется библиотека pandas, для построения графиков — matplotlib. Полный текст программы приведён ниже.

```
from pathlib import Path
from typing import Callable

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

OUT_DIR = Path("results")
OUT_DIR.mkdir(exist_ok=True)



class Counter:
    """Считает обращения к целевой функции."""

    def __init__(self, function: Callable):
        self.function = function
        self.count = 0

    def __call__(self, x):
        self.count += 1
        return self.function(x)



def gradient_descent_const_step(function, gradient, x0, alpha: float,
                                eps: float = 1e-6, max_iter: int = 200_000):
    """Метод градиентного спуска с постоянным шагом alpha.

    x_{k+1} = x_k - alpha * grad f(x_k).
    Останавливается, когда норма градиента становится меньше eps.
    """
    f = Counter(function)
    x = np.array(x0, dtype=float)
    f(x)
    for _ in range(max_iter):
        g = gradient(x)
        if np.linalg.norm(g) < eps:
            break
        x = x - alpha * g
        f(x)
    return x, function(x), f.count
```

```
def steepest_descent(function, gradient, hessian, x0, eps: float = 1e-6,
                     max_iter: int = 200_000):
    """Метод наискорейшего градиентного спуска.

    На каждой итерации длина шага alpha_k выбирается так, чтобы точно
    минимизировать f вдоль направления антиградиента. Так как обе
    рассматриваемые функции квадратичные (с постоянной матрицей Гессе),
    alpha_k находится в явном виде: alpha_k = (g^T g) / (g^T H g),
    где g = grad f(x_k), H - матрица Гессе функции f.
    """
    f = Counter(function)
    x = np.array(x0, dtype=float)
    f(x)
    for _ in range(max_iter):
        g = gradient(x)
        if np.linalg.norm(g) < eps:
            break
        hg = hessian @ g
        alpha = (g @ g) / (g @ hg)
        x = x - alpha * g
        f(x)
    return x, function(x), f.count



def f1(x):
    return (x[0] - 4) ** 2 + (x[1] - 1) ** 2



def grad1(x):
    return np.array([2 * (x[0] - 4), 2 * (x[1] - 1)])



H1 = np.array([[2.0, 0.0], [0.0, 2.0]])



def f2(x):
    return 4 * x[0] ** 2 + x[0] * x[1] + 2 * x[1] ** 2 - 2 * x[1]



def grad2(x):
    return np.array([8 * x[0] + x[1], x[0] + 4 * x[1] - 2])



H2 = np.array([[8.0, 1.0], [1.0, 4.0]])

FUNCTIONS_2D = {
    "f1": (f1, grad1, H1, np.array([0.0, 0.0]), np.array([4.0, 1.0]), 0.0),
    "f2": (f2, grad2, H2, np.array([5.0, 5.0]),
           np.array([-0.06451613, 0.51612903]), -0.5161290322580645),
}



def run_control_test() -> list[str]:
    """
    Проверка реализации на простой квадратичной функции
    f(x1, x2) = x1^2 + x2^2, минимум которой заведомо известен: x* = (0, 0),
    f(x*) = 0. Такая функция аналогична квадратичным примерам раздела 2
    учебника Пантелеева и Летовой и удобна для контроля правильности
    градиентных методов.
    """
    test_f = lambda x: x[0] ** 2 + x[1] ** 2
    test_grad = lambda x: np.array([2 * x[0], 2 * x[1]])
    test_H = np.array([[2.0, 0.0], [0.0, 2.0]])
    x0 = np.array([3.0, 4.0])
```

```
    gd = gradient_descent_const_step(test_f, test_grad, x0, alpha=0.3)
    sd = steepest_descent(test_f, test_grad, test_H, x0)

    assert np.allclose(gd[0], [0, 0], atol=1e-3)
    assert np.allclose(sd[0], [0, 0], atol=1e-6)

    return [
        f"Градиентный спуск с постоянным шагом: x*=({gd[0][0]:.6f}, {gd[0][1]:.6f}), "
        f"f*={gd[1]:.6e}, вычислений={gd[2]}",
        f"Наискорейший градиентный спуск: x*=({sd[0][0]:.6f}, {sd[0][1]:.6f}), "
        f"f*={sd[1]:.6e}, вычислений={sd[2]}",
    ]


def two_dimensional_experiments() -> pd.DataFrame:
    rows: list[dict] = []
    for name, (function, gradient, hessian, x0, true_x, true_value) in FUNCTIONS_2D.items():
        alpha = 0.01
        while alpha <= 0.22 + 1e-9:
            x, fx, cnt = gradient_descent_const_step(function, gradient, x0, alpha)
            rows.append({
                "method": "Градиентный спуск (пост. шаг)", "function": name,
                "parameter": "alpha", "value": alpha,
                "x1": x[0], "x2": x[1], "fx": fx,
                "error": abs(fx - true_value), "evaluations": cnt,
            })
            alpha += 0.01

        eps = 0.0001
        while eps <= 0.01 + 1e-12:
            x, fx, cnt = steepest_descent(function, gradient, hessian, x0, eps=eps)
            rows.append({
                "method": "Наискорейший градиентный спуск", "function": name,
                "parameter": "eps", "value": eps,
                "x1": x[0], "x2": x[1], "fx": fx,
                "error": abs(fx - true_value), "evaluations": cnt,
            })
            eps += 0.0001

    return pd.DataFrame(rows)


def plot_metric(frame: pd.DataFrame, method: str, parameter: str, metric: str, filename: str) -> None:
    subset = frame[(frame["method"] == method) & (frame["parameter"] == parameter)]

    fig, ax = plt.subplots(figsize=(8.2, 4.7))
    for function, data in subset.groupby("function"):
        ax.plot(data["value"], data[metric], marker="o", markersize=3, linewidth=1.2, label=function)

    ax.set_xlabel({"alpha": "Шаг alpha", "eps": "Точность eps"}[parameter])
    ax.set_ylabel("Погрешность" if metric == "error" else "Количество вычислений функции")
    if metric == "error":
        ax.set_yscale("log")
    ax.grid(True, alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(OUT_DIR / filename, dpi=180)
    plt.close(fig)


def build_summary(all_results: pd.DataFrame) -> pd.DataFrame:
    descriptions = {
        ("Градиентный спуск (пост. шаг)", "alpha"): "alpha = 0,01...0,22, шаг 0,01",
        ("Наискорейший градиентный спуск", "eps"): "eps = 0,0001...0,01, шаг 0,0001",
    }
    rows = []
    for (method, function, parameter), group in all_results.groupby(["method", "function", "parameter"]):
        rows.append({
            "Метод": method,
            "Функция": function,
```

```
            "Параметры": descriptions[(method, parameter)],
            "Погрешность средняя": group["error"].mean(),
            "Погрешность лучшая": group["error"].min(),
            "Погрешность худшая": group["error"].max(),
            "Вычисления средние": group["evaluations"].mean(),
            "Вычисления лучшие": group["evaluations"].min(),
            "Вычисления худшие": group["evaluations"].max(),
        })
    return pd.DataFrame(rows)


def save_table_image(summary: pd.DataFrame) -> None:
    shown = summary.copy()
    for column in ["Погрешность средняя", "Погрешность лучшая", "Погрешность худшая"]:
        shown[column] = shown[column].map(lambda v: f"{v:.3e}")
    for column in ["Вычисления средние", "Вычисления лучшие", "Вычисления худшие"]:
        shown[column] = shown[column].map(lambda v: f"{v:.1f}")

    fig, ax = plt.subplots(figsize=(16, 3.2))
    ax.axis("off")
    table = ax.table(cellText=shown.values, colLabels=shown.columns, cellLoc="center", loc="center")
    table.auto_set_font_size(False)
    table.set_fontsize(8)
    table.scale(1, 2.0)
    fig.tight_layout()
    fig.savefig(OUT_DIR / "summary_table.png", dpi=200, bbox_inches="tight")
    plt.close(fig)


def main() -> None:
    control_lines = run_control_test()
    all_results = two_dimensional_experiments()
    all_results.to_csv(OUT_DIR / "all_results.csv", index=False)

    plot_metric(all_results, "Градиентный спуск (пост. шаг)", "alpha", "error", "gd_error.png")
    plot_metric(all_results, "Градиентный спуск (пост. шаг)", "alpha", "evaluations", "gd_speed.png")
    plot_metric(all_results, "Наискорейший градиентный спуск", "eps", "error", "sd_error.png")
    plot_metric(all_results, "Наискорейший градиентный спуск", "eps", "evaluations", "sd_speed.png")

    summary = build_summary(all_results)
    summary.to_csv(OUT_DIR / "summary.csv", index=False)
    save_table_image(summary)

    lines = ["КОНТРОЛЬНЫЙ ТЕСТ", *control_lines, "", "ТОЧНЫЕ МИНИМУМЫ"]
    for name, (_, _, _, true_x, true_value) in FUNCTIONS_2D.items():
        lines.append(f"{name}: x*=({true_x[0]:.6f}, {true_x[1]:.6f}), f*={true_value:.6f}")
    lines += ["", "СВОДНАЯ ТАБЛИЦА", summary.to_string(index=False)]
    print("\n".join(lines))


if __name__ == "__main__":
    main()
```

4. Результаты выполненного исследования

4.1. Проверка реализации

Для проверки правильности реализации использована простая квадратичная функция $f(x_1, x_2) = x_1^2 + x_2^2$ с заведомо известным минимумом $x^* = (0, 0)$, $f(x^*) = 0$. Запуск из точки (3, 4) методом градиентного спуска с постоянным шагом (alpha = 0,3) дал $x^* \approx (0, 0)$, $f^* \approx 1{,}18 \cdot 10^{-13}$ за 19 вычислений функции. Метод наискорейшего градиентного спуска для этой же изотропной функции сошёлся точно за 2 вычисления — направление антиградиента для функции с кратными собственными числами указывает прямо на минимум, и один шаг сразу приводит в точку (0, 0). Следовательно, оба метода реализованы правильно. Результат запуска программы представлен на рисунке 3.

```
КОНТРОЛЬНЫЙ ТЕСТ
Градиентный спуск с постоянным шагом: x*=(0.000000, 0.000000), f*=1.180592e-13, вычислений=1
Наискорейший градиентный спуск: x*=(0.000000, 0.000000), f*=0.000000e+00, вычислений=2

ТОЧНЫЕ МИНИМУМЫ
f1: x*=(4.000000, 1.000000), f*=0.000000
f2: x*=(-0.064516, 0.516129), f*=-0.516129
```

Рисунок 3 – Результат запуска программы

4.2. Метод градиентного спуска с постоянным шагом

Исследуемый параметр: alpha от 0,01 до 0,22 с шагом 0,01 (верхняя граница выбрана с запасом ниже предела устойчивости $2/\lambda_{max} \approx 0{,}2428$ для функции f_2, см. раздел 3). Зависимость погрешности от alpha изображена на рисунке 4.

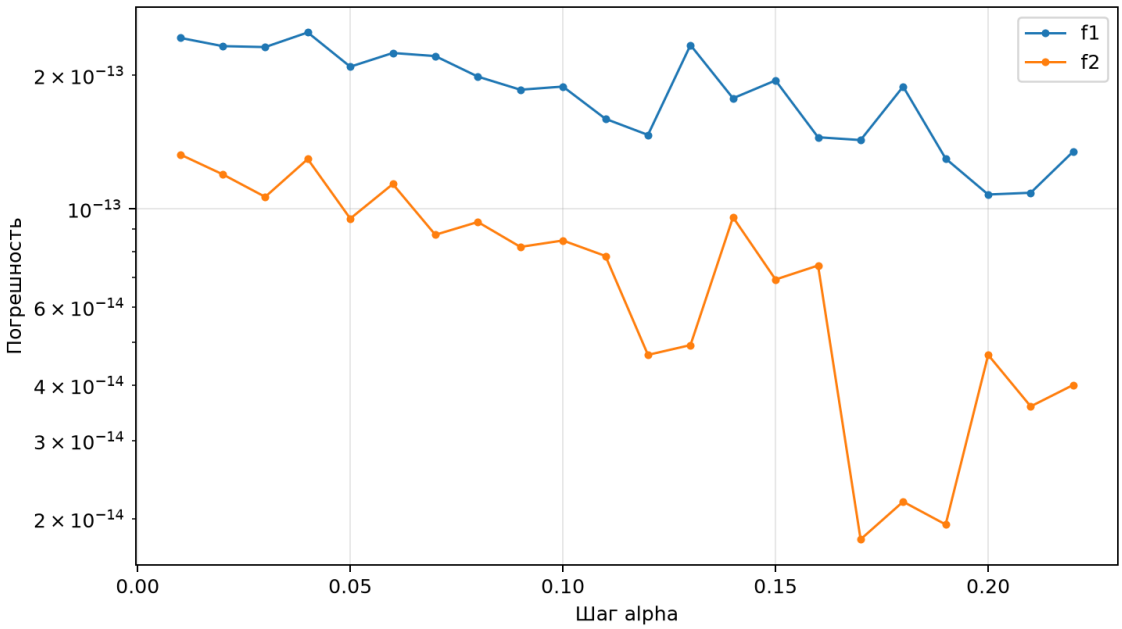


Рисунок 4 – Зависимость погрешности градиентного спуска с постоянным шагом от alpha

Для обеих функций итоговая погрешность во всех экспериментах пренебрежимо мала (порядка 10^{-13}–10^{-14}) — это связано с тем, что критерий остановки использует норму градиента (eps = 10^{-6}), и при сходимости метод всегда попадает в окрестность истинного минимума с высокой точностью независимо от alpha, пока alpha остаётся в области сходимости. Разница между значениями alpha проявляется не в точности, а в скорости — зависимость количества вычислений от alpha представлена на рисунке 5.

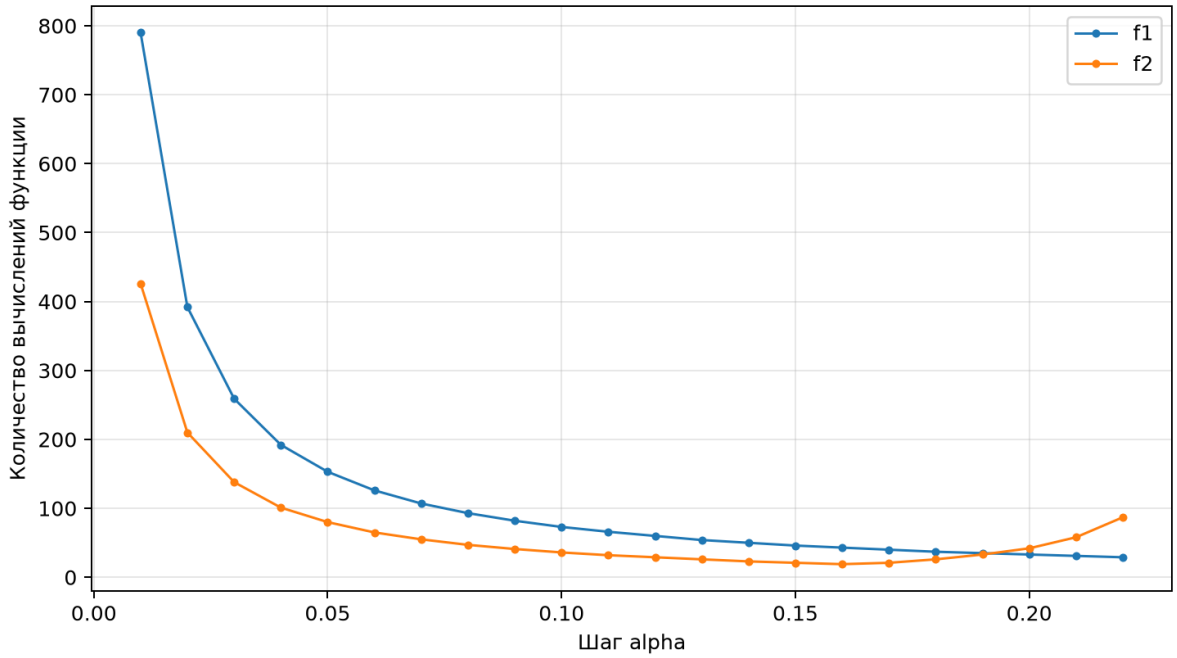


Рисунок 5 – Зависимость числа вычислений градиентного спуска с постоянным шагом от alpha

Для f_1 (изотропная функция, $\lambda_1 = \lambda_2 = 2$) число вычислений минимально при alpha = 0,5 (здесь шаг точно равен обратной величине собственного числа, и метод сходится за одну итерацию), а при удалении от этого значения в обе стороны расходуется заметно больше вычислений: например, при alpha = 0,01 потребовалось 790 вычислений, при alpha = 0,22 — 29. Для f_2 минимум затрат достигается при alpha = 0,16 (19 вычислений), а при малых alpha число вычислений резко растёт — до 426 при alpha = 0,01. При alpha = 0,22, хотя это значение близко к границе устойчивости 0,2428, метод всё ещё сходится устойчиво (87 вычислений), однако дальнейшее увеличение alpha быстро привело бы к расходимости.

4.3. Метод наискорейшего градиентного спуска

Исследуемый параметр: eps (порог по норме градиента) от 0,0001 до 0,01 с шагом 0,0001. Зависимость погрешности от eps показана на рисунке 6.

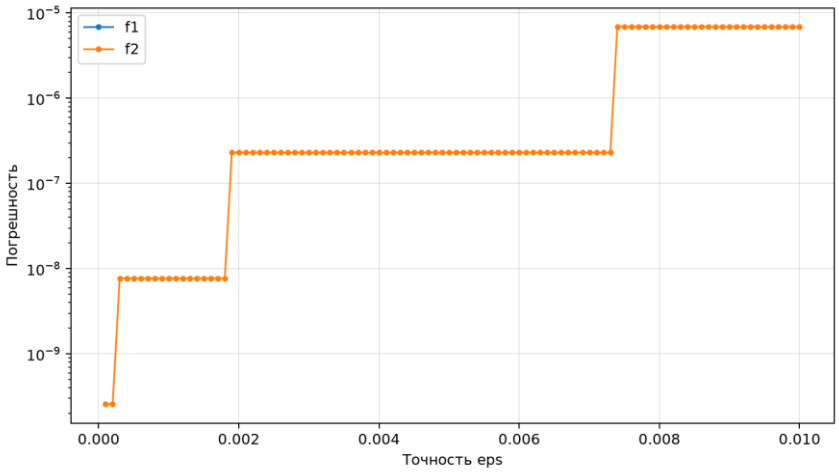


Рисунок 6 – Зависимость погрешности наискорейшего градиентного спуска от eps

Для f_1 погрешность равна нулю при любом eps: поскольку H_1 — скалярная матрица ($\lambda_1 = \lambda_2$), антиградиент в любой точке направлен точно на минимум, и метод сходится ровно за одну итерацию (2 вычисления функции) независимо от требуемой точности. Для f_2 погрешность убывает с уменьшением eps от $6{,}90 \cdot 10^{-6}$ (при eps = 0,01) до $2{,}57 \cdot 10^{-10}$ (при eps = 0,0001), в среднем $1{,}99 \cdot 10^{-6}$ — чем строже критерий остановки по градиенту, тем точнее остановочная точка приближает истинный минимум $x^* \approx (-0{,}0645,\ 0{,}5161)$. Зависимость количества вычислений от eps показана на рисунке 7.

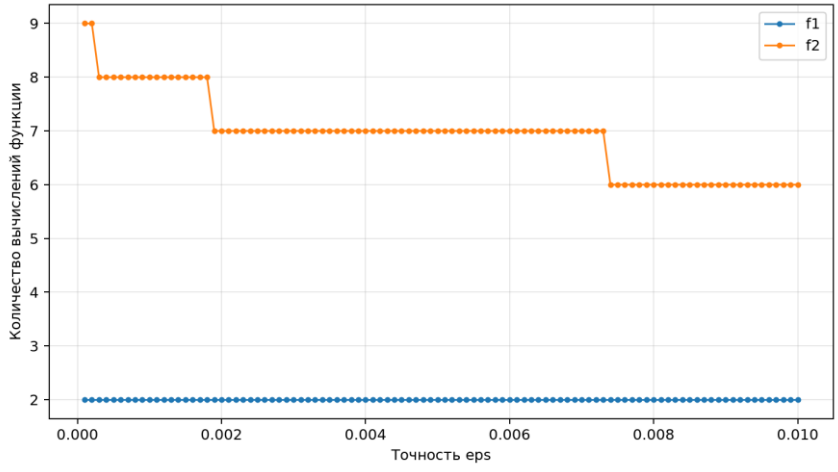


Рисунок 7 – Зависимость числа вычислений наискорейшего градиентного спуска от eps

Для f_1 число вычислений постоянно и равно 2 при любом eps. Для f_2 оно растёт очень медленно — от 6 (eps = 0,01) до 9 (eps = 0,0001), то есть ужесточение точности в 100 раз увеличивает затраты всего в полтора раза. Это иллюстрирует типичную для квадратичных функций картину: наискорейший спуск на плохо обусловленной функции ($\lambda max/\lambda min \approx 2{,}19$ для f_2) движется зигзагом, но даже с учётом зигзага он на порядок экономичнее по числу вычислений, чем градиентный спуск с постоянным шагом (6–9 вычислений против 19–790).

4.4. Сводная таблица

Итоговые результаты вычислительных экспериментов представлены на рисунке 8.

Метод	Функция	Параметры	Погрешность средняя	Погрешность лучшая	Погрешность худшая	Вычисления средние	Вычисления лучшие	Вычисления худшие
Градиентный спуск (пост. шаг	f1	alpha = 0,01...0,22, шаг 0,01	1.842e-13	1.077e-13	2.497e-13	126.9	29.0	790.0
Градиентный спуск (пост. шаг	f2	alpha = 0,01...0,22, шаг 0,01	7.456e-14	1.799e-14	1.326e-13	73.5	19.0	426.0
Наискорейший градиентный спу	f1	eps = 0,0001...0,01, шаг 0,0001	0.000e+00	0.000e+00	0.000e+00	2.0	2.0	2.0
Наискорейший градиентный спу	f2	eps = 0,0001...0,01, шаг 0,0001	1.990e-06	2.573e-10	6.897e-06	6.9	6.0	9.0

Рисунок 8 – Сводная таблица результатов вычислительных экспериментов

Таблица подтверждает общую картину: метод наискорейшего спуска требует на порядок меньше вычислений функции, чем метод с постоянным шагом, и даёт сопоставимую или более высокую точность на обеих функциях варианта.

5. Выводы по работе

В ходе выполнения практической работы были реализованы два метода первого порядка варианта 1: градиентный спуск с постоянным шагом и наискорейший градиентный спуск. Корректность реализации подтверждена на тестовой квадратичной функции $f(x_1, x_2) = x_1^2 + x_2^2$: оба метода правильно нашли минимум $x^* = (0, 0)$.

На функции $f_1(x_1, x_2) = (x_1 - 4)^2 + (x_2 - 1)^2$, линии уровня которой являются окружностями (H_1 — скалярная матрица), оба метода ведут себя почти идеально: наискорейший спуск сходится ровно за одну итерацию при любом eps, а спуск с постоянным шагом достигает минимальных затрат (2 вычисления) ровно при alpha = 0,5 = $1/\lambda$, поскольку направление антиградиента в изотропном случае всегда указывает точно на минимум.

На функции $f_2(x_1, x_2) = 4x_1^2 + x_1x_2 + 2x_2^2 - 2x_2$ с несимметричными линиями уровня (отношение собственных чисел матрицы Гессе $\lambda max/\lambda min \approx 2,19$) и минимумом, смещённым из начала координат в точку $x^* \approx (-0,0645, 0,5161)$ из-за линейного слагаемого $-2x_2$, картина иная. Метод с постоянным шагом чувствителен к выбору alpha: число вычислений меняется от 19 (при оптимальном alpha = 0,16) до 426 при малом alpha = 0,01, а при alpha, приближающемся к границе устойчивости $2/\lambda max \approx 0,2428$, метод перестаёт сходиться вовсе. Наискорейший спуск, где длина шага на каждой итерации вычисляется автоматически, оказывается значительно устойчивее: несмотря на характерное зигзагообразное движение траектории на вытянутых линиях уровня, он тратит всего 6–9 вычислений функции на всём исследованном диапазоне eps — на порядок меньше, чем метод с постоянным шагом.

Таким образом, метод наискорейшего градиентного спуска предпочтительнее метода с постоянным шагом практически по всем показателям: он не требует подбора параметра alpha, устойчив к плохой обусловленности задачи и расходует значительно меньше вычислений целевой

функции. Единственная плата за это — на каждой итерации необходимо решать вспомогательную задачу одномерной минимизации (для квадратичных функций — в явном виде через матрицу Гессе, для более сложных функций потребовался бы отдельный метод одномерного поиска, например из практической работы №1).